

Bhargavi Rengarajan

+1(951)334-6504 | bhargavirengarajan21@gmail.com | [linkedin.com/in/bhargavi-r21](https://www.linkedin.com/in/bhargavi-r21) | <https://github.com/bhargavirengarajan21> | Chicago, IL

TECHNICAL SKILLS

Languages: Java, TypeScript, JavaScript, Python, Go, SQL, C/C++, R, HTML5, CSS3

Frontend: React, Next.js, Redux, React Context, GraphQL, Tailwind CSS, Material-UI, SCSS, Storybook, Webpack

Backend: Node.js, Express.js, Spring Boot (WebFlux), Flask, RESTful API, OpenAPI, Kafka, Azure Service Bus, Redis, MongoDB, PostgreSQL

Cloud & DevOps: AWS (EC2), Azure (Pipelines), Google Cloud Platform, Docker, Kubernetes, GitHub Actions, New Relic, Git

Testing & Accessibility: Jest, React Testing Library, Cypress, JUnit, Lighthouse

AI & Data Science: Claude, Model Context Protocol (MCP), Gemini, Ollama, XGBoost, CuPy, PySpark, Hadoop, pandas, NumPy, Matplotlib

Competencies: Frontend development, Agile methodologies, Scrum, Full-Stack, Distributed Systems, Microservices, Data Structures, Algorithms, Server Side Rendering, Macro Frontend, Microservices

EXPERIENCE

Rocket Mortgage

Remote, USA

Software Development Engineer II

March 2025 – Present

- Refactored the draw-eligibility logic to maintain data integrity, processing requests on a 1-hour interval in a Java Spring Boot microservice, eliminating duplicate approvals.
- Designed and prototyped an MCP-based GenAI tool connecting Claude to New Relic through an Express MCP server and a Next.js frontend, enabling plain-language queries over production monitoring data.
- Fortified the loan-servicing event pipeline over Azure Service Bus by catching and marking failed events, so 100% of previously silent failures are now handled instead of lost.
- Achieved 100% recovery of failed callback events with a Kafka event listener that retries 3 times before routing to a dead-letter queue, deployed through Docker and Azure Pipelines.
- Developed a RESTful PATCH endpoint to cancel a payment that coordinates 3 downstream side-effect calls, completing the full cancel-and-update in 2.5s.
- Designed robust schema for payment audit trail in SQL that records every transaction update, decreased data corruption and redundancy by 50%.
- Advocated for strict input validation at the contract boundary using OpenAPI and a global exception handler, tightening field-length limits so invalid data is rejected before it reaches the service.
- Standardized error handling and response format across backend services in Spring Boot REST, replacing inconsistent formats with one contract and cutting debugging time 35%.
- Monitored production servicing jobs by adding structured logging and New Relic alerts into Teams, cutting on-call noise 83% so the rotation focuses on real incidents.
- Verified consistent service health across all deployment environments by owning the observability configuration and infrastructure-as-code, so every environment reports the same way.
- Developed Node.js middleware that runs dependent eligibility checks for 4M applications in under 180ms and removed 95% of manual review.
- Created a React dashboard component backed by server-side Redis caching with a 24-hour TTL, cutting an external vendor call from 10,000 requests a day to 1 per user.
- Integrated Microsoft Omnichannel live chat, which runs as its own microservice, across the React web application so customers get real-time answers on marketing pages.
- Maintained a reusable React component library shared across 5 brand applications, keeping 20+ components consistent and removing duplicate UI code.
- Owned observability for scheduled-job services by configuring New Relic alerts, reducing debugging time from 15 minutes to 2 minutes.
- Automated testing, linting, and snapshot generation in GitHub Actions CI/CD, reducing manual deployment effort 40%.
- Expanded the Spring Boot and Kafka messaging service to deliver across multiple brands, growing a single-brand pipeline into multi-brand delivery that lifted reach 65% and engagement 15%.

- Served as a primary contact for distributed cross-timezone teams, unblocking release dependencies and cutting handoff delays by 50%.
- Owned end-to-end delivery of a full-stack application: a Next.js frontend with Node.js middleware that schedules page rendering and fetches data and config from GCP, unblocking a stalled release and doubling deployment speed.

Mr. Cooper

Chennai, India

Software Development Engineer II

April 2022 – September 2023

- Led a banner-automation workflow, building a Node.js and Webpack code-generation pipeline with an ERB/EJS transpiler that replaced manual scaffolding and cleared a 200-story backlog, cutting banner creation from 3 engineer-days to 30 minutes.
- Published a typed, reusable API client in TypeScript with Axios interceptors, keep-alive connections, and Redis, standardizing fragmented integration across 5 apps with 10 endpoints and zero connection leaks.
- Engaged with the tech lead to migrate the legacy React app to Next.js for server-side rendering, improving page load performance 60%.
- Resolved duplicated state and layout code across 40+ pages by refactoring shared layouts with design input, so a single layout change now ships from one place.
- Revamped the styling foundation into reusable SCSS mixins, palette, and utilities for white-label brands, collapsing 50+ duplicated style files into one across 2 brands.
- Refactored Dockerfiles into multi-stage builds, reducing container image size 40%.
- Accelerated React page loads by building a scheduled microservice cronjob that pulls third-party mortgage rate data into Redis on a 24-hour refresh, so pages read ready data instead of waiting on the external call.
- Launched a personalized loan officer page in React and GraphQL with filtering and sorting, improving customer engagement 25%.
- Measured marketing-funnel behavior by adding Google Analytics across 15+ pages, turning a no-data gap into conversion insight.
- Solved slow dashboard reads by profiling and rewriting MongoDB aggregation queries behind Express and Redux, cutting latency from 560ms to 100ms for 9,000+ users.

Mr. Cooper

Chennai, India

Software Development Engineer I

July 2020 – March 2022

- Addressed ADA and WCAG accessibility failures across 100+ pages, working with the team to add ARIA and enforce checks through eslint-plugin-jsx-a11y with Lighthouse and WAVE audits.
- Validated component behavior across unit, integration, and end-to-end layers with Jest, React Testing Library, Cypress, and Husky, raising coverage on 40+ components and cutting regressions 30%.
- Streamlined the application form into a React multistep flow that preloads the user's saved data through Node.js and Redux, cutting repeated data entry.
- Adapted the marketing pages for internationalization and localization across roughly 5 brands, so each brand serves content in its own locale from one codebase.
- Built and documented a shared UI component library in React, Storybook, and Context API, adopted across 4 applications to end fragmented UI work.
- Volunteered to fix broken local tooling for the team, adding a Node.js and Express proxy that unblocked engineers' local environments.

PROJECTS

Git Commit Tool | Open Source | *Node.js, Ollama, Docker*

- Engineered an on-device Git subcommand using Node.js and Ollama, achieving 168× size reduction and 381 downloads.

Netflix-Inspired Portfolio | *Gemini, TypeScript, AWS EC2, Tailwind, GitHub Actions*

- Developed a Netflix-style portfolio with a Gemini chatbot, improving recruiter engagement 30% and reducing bounce rate 20%.

Distributed Cloud Logging | *Golang, Kubernetes, Blockchain*

- Developed a serverless, tamper-resistant cloud logging system using Golang services deployed on Kubernetes, enabling real-time log capture and reducing debugging time by 60%.

Air Pollution Prediction | Research Project | *Python, Flask, XGBoost*

- Developed a full-stack Flask PM10 prediction system using XGBoost, achieving 95% cross-validated accuracy.

Electrostatic Dithering & Visualization | *Python, CuPy, Google Colab*

- Engineered GPU-accelerated electrostatic dithering algorithms and visualized force distributions to improve pattern uniformity and reduce rendering artifacts.

US Accident Zone Visualization | *React, PySpark, Hadoop, React Maps*

- Developed a geospatial accident-zone visualization platform with a React frontend and PySpark processing on Hadoop-backed datasets to identify high-risk hotspots.

EDUCATION

University of California, Riverside

Master of Science in Computer Engineering

Riverside, California

September 2023 – March 2025

Thiagarajar College of Engineering

Bachelor of Engineering in Computer Science

Tamil Nadu, India

August 2016 – April 2020